

GPT-5.6: The Complete Guide

From GPT-5's Launch Leap to GPT-5.6 Today

Lucas Soares

August 2026

Table of Contents

Act I — GPT-5: The Original Leap (Aug 2025)

- Launch timeline & core innovations
- Why it was a big deal: the benchmarks
- System Card: router, safety, honest problems

Bridge — One Year Later: GPT-5 → GPT-5.6

Act II — GPT-5.6: Where We Are Today (Jul 2026)

- What actually changed, and what didn't
- Honest assessment of today's model
- Building with GPT-5.6: API, Agents SDK, prompting

Act I

GPT-5: The Original Leap

August 2025

GPT-5 Launch Timeline

- **Launch Date:** August 7, 2025
- **Major Breakthrough:** first unified system combining reasoning, multimodal input, and task execution — replacing the old pick-a-model-yourself menu (GPT-4o, GPT-4.5, o3, o3-pro, o4-mini, GPT-4.1...) with one model, GPT-5
- **Availability:** ChatGPT Free/Plus/Pro/Team at launch, API same day, Enterprise/Edu shortly after

GPT-5 Core Innovations (At Launch)

- **Unified Architecture** — one system, multiple specialized components, instead of a model-picker menu
- **Smart Router** — automatically selects the best internal model for each task
- **Reduced Hallucinations** — 45% fewer factual errors than GPT-4o
- **Enhanced Tool Use** — reliably chains dozens of tool calls
- **Multimodal Excellence** — native text, image, audio, and video support

GPT-4o
Great for most tasks

GPT-4.5
Good for writing

o3
Uses advanced reasoning

o3-pro
Best at reasoning

o4-mini
Fastest at advanced reasoning

o4-mini-high
Great at coding and reasoning

GPT-4.1
Great for quick coding

GPT-4.1-mini
Faster for everyday tasks



GPT-5 ✓
Flagship model

After: just "GPT-5"

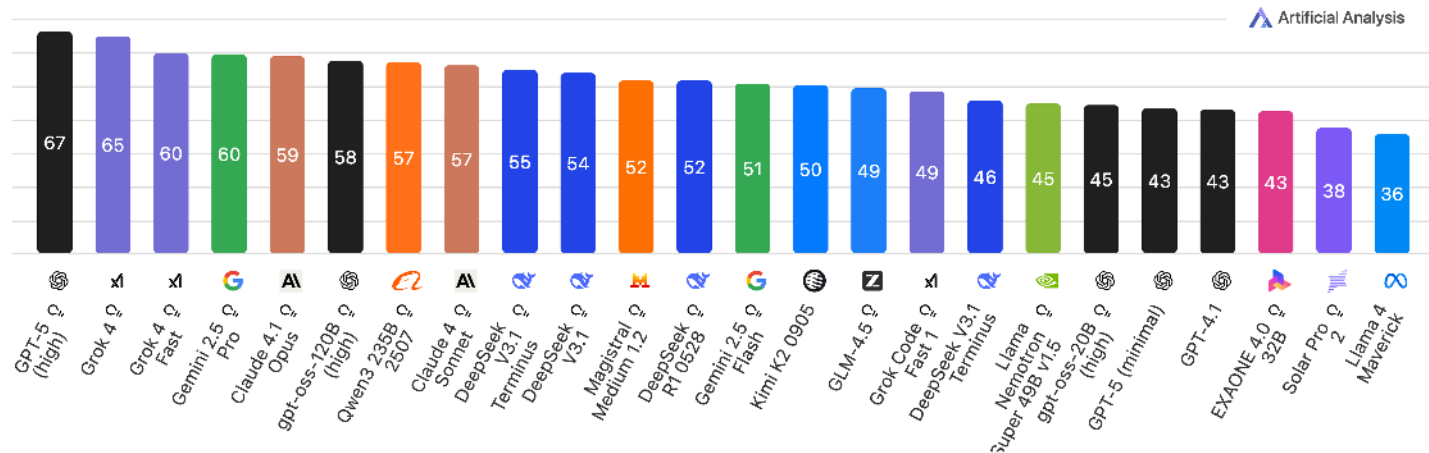
Before: pick from 8 models

Why GPT-5 Was a Big Deal: The Benchmarks

GPT-5: SOTA on the Artificial Analysis Intelligence Index

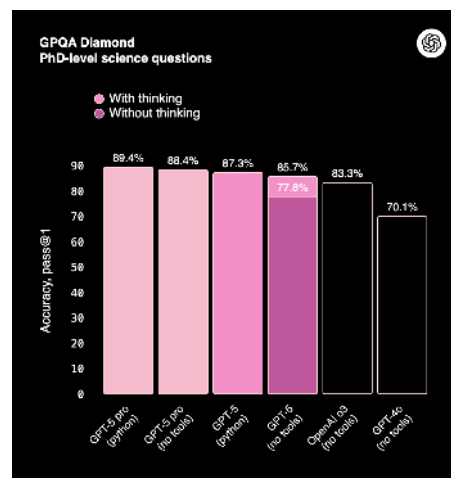
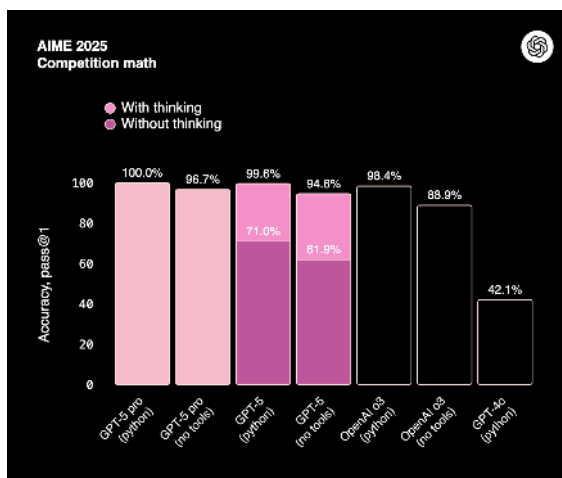
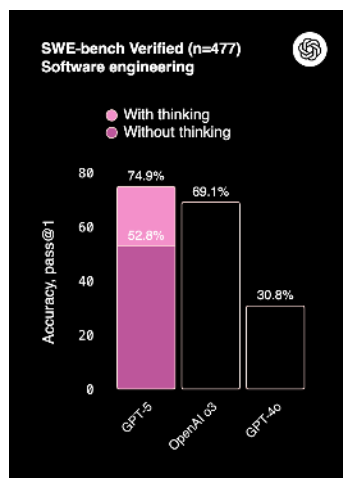
Artificial Analysis Intelligence Index

Artificial Analysis Intelligence Index v3.0 incorporates 10 evaluations: MMLU-Pro, GPQA Diamond, Humanity's Last Exam, LiveCodeBench, SciCode, AIME 2025, IFBench, AA-LCR, Terminal-Bench Hard, τ^2 -Bench Telecom



Source: <https://openai.com/index/introducing-gpt-5/>

Across the Board: Frontier Results at Launch



Also SOTA at launch on: FrontierMath (expert math), Aider Polyglot (multilang code editing), Humanity's Last Exam, tau2-bench (function calling), and multimodal visual/video reasoning.

Overall Performance Summary

Mathematics: 94.6% on AIME 2025 (without tools)

Coding: 74.9% on SWE-bench Verified, 88% on Aider Polyglot

Multimodal: 84.2% on MMMU understanding

Healthcare: 46.2% on HealthBench Hard (vs 31.6% for OpenAI o3)

Reduced errors, not just raw scores: 45% fewer factual errors than GPT-4o, 80% fewer with thinking mode vs OpenAI o3 — and matched/beat o3 quality while spending less time thinking per answer

Ok, great the model does well on a lot of benchmarks, but what is new about it?

GPT-5 System Card

Historical Architecture

Notes on GPT-5's System Card

Real-time Router

- Intelligently decided which model to use
- Based on conversation type, complexity, and tools needed

Smart and Fast Model (`gpt-5-main`)

- Handled most everyday questions efficiently
- Optimized for speed and general capability

Deeper Reasoning Model (`gpt-5-thinking`)

- Tackled complex problems requiring deep thought
- Produced long internal chains of reasoning, could refine strategy mid-answer

Naming It: System Card vs. API vs. What It Replaced

SYSTEM CARD NAME	API ALIAS
gpt-5-thinking	gpt-5
gpt-5-thinking-mini	gpt-5-mini
gpt-5-thinking-nano	gpt-5-nano
gpt-5-main	gpt-5-chat-latest
gpt-5-main-mini	[not available via API]

Previous model	GPT-5 model
GPT-4o	gpt-5-main
GPT-4o-mini	gpt-5-main-mini
OpenAI o3	gpt-5-thinking
OpenAI o4-mini	gpt-5-thinking-mini
GPT-4.1-nano	gpt-5-thinking-nano
OpenAI o3 Pro	gpt-5-thinking-pro

Safety Approach

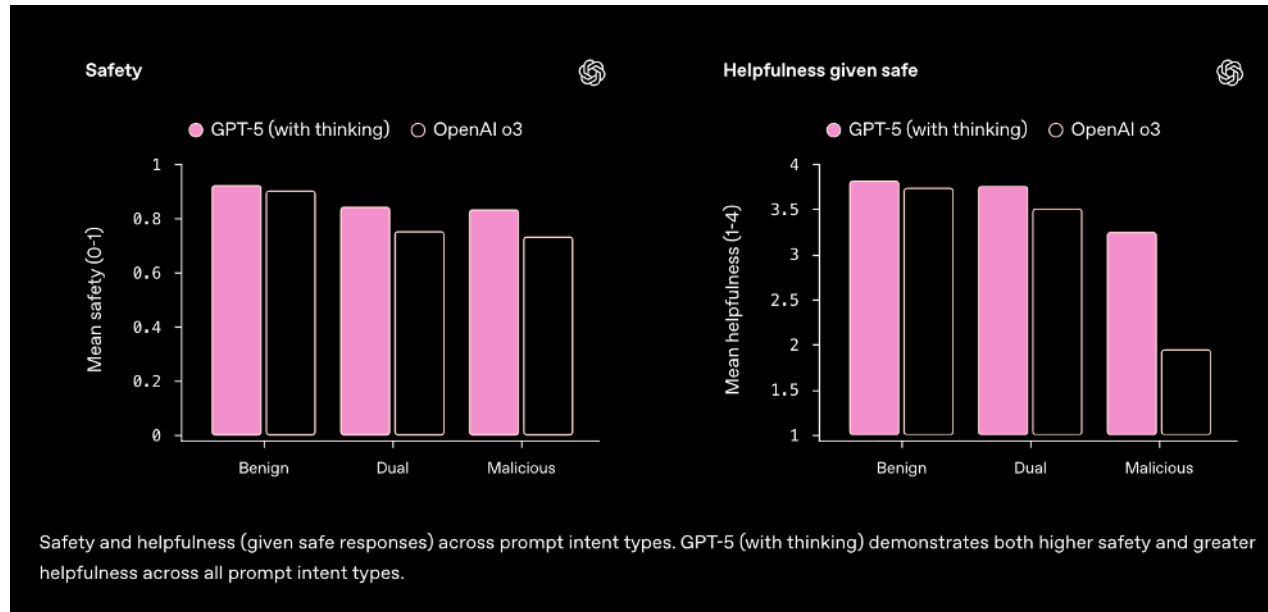
Traditional Safety Approach

- Binary refusal (allow/deny)
- Overly cautious, often blocks helpful answers
- Lacks nuance for complex or dual-use queries

GPT-5 Safety-Helpfulness ("Safe Completions")

- Nuanced, context-aware responses
- Balances safety with helpfulness
- Enables more informative answers while respecting policy

Safety vs. Helpfulness, Measured



Key Benefits of Safe-Completions

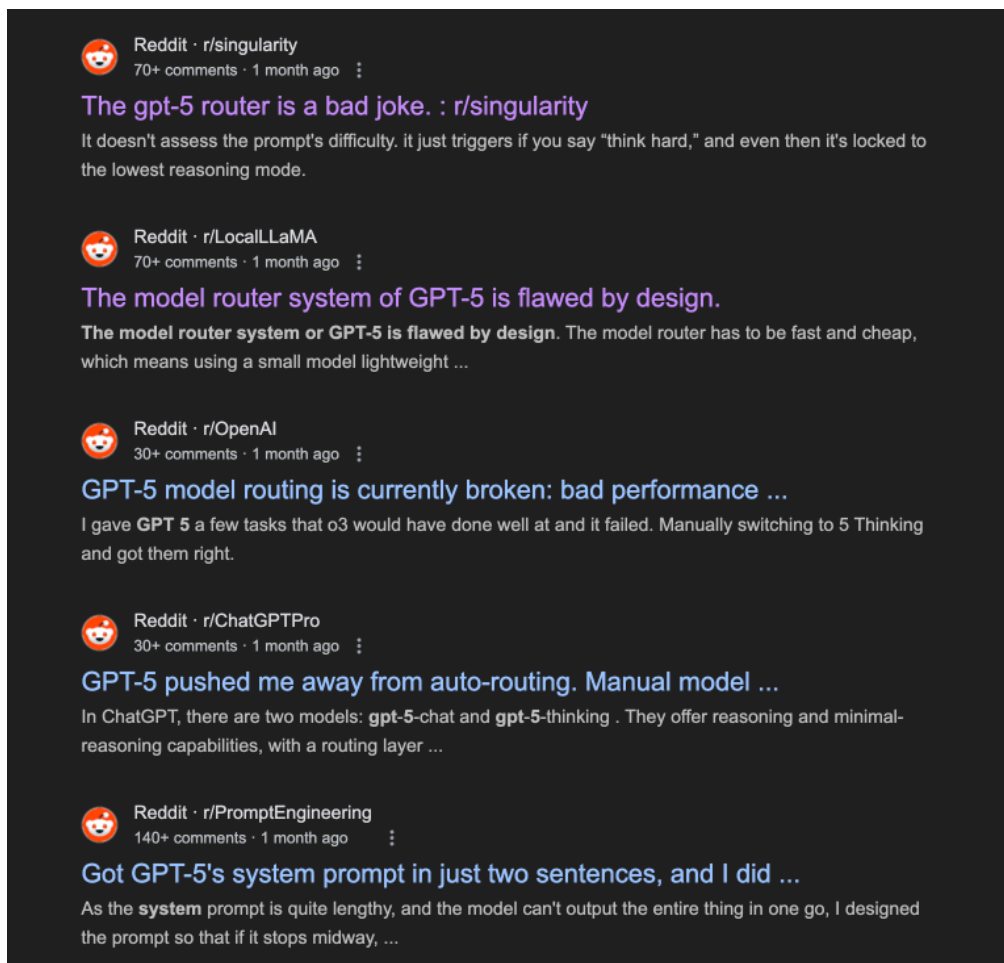
- **Maximizes helpfulness** while maintaining safety policy constraints
- **Particularly effective** for dual-use cases (biology, cybersecurity)
- **Nuanced responses** instead of blanket refusals
- **Still the industry standard today** — GPT-5.6 ships on the same safe-completions training approach

Honest Assessment: GPT-5 at Launch

The Rollout Was Rockier Than the Benchmarks Suggest

- **Hype vs. reality** — pitched as a possible AGI-adjacent leap (Altman: like the jump to Retina-display iPhones)
- **Press verdict** — "incremental, not revolutionary": an expectations gap, not a capability failure
- **GPT-4o backlash** — sudden retirement of the old model drew real user pushback; OpenAI later called it "a miss"

The Router Was the Biggest Complaint



Reddit · r/singularity
70+ comments · 1 month ago

The gpt-5 router is a bad joke. : r/singularity

It doesn't assess the prompt's difficulty. it just triggers if you say "think hard," and even then it's locked to the lowest reasoning mode.

Reddit · r/LocalLLaMA
70+ comments · 1 month ago

The model router system of GPT-5 is flawed by design.

The model router system or GPT-5 is flawed by design. The model router has to be fast and cheap, which means using a small model lightweight ...

Reddit · r/OpenAI
30+ comments · 1 month ago

GPT-5 model routing is currently broken: bad performance ...

I gave GPT 5 a few tasks that o3 would have done well at and it failed. Manually switching to 5 Thinking and got them right.

Reddit · r/ChatGPTPro
30+ comments · 1 month ago

GPT-5 pushed me away from auto-routing. Manual model ...

In ChatGPT, there are two models: gpt-5-chat and gpt-5-thinking . They offer reasoning and minimal-reasoning capabilities, with a routing layer ...

Reddit · r/PromptEngineering
140+ comments · 1 month ago

Got GPT-5's system prompt in just two sentences, and I did ...

As the system prompt is quite lengthy, and the model can't output the entire thing in one go, I designed the prompt so that if it stops midway, ...

Source: r/singularity, r/LocalLLaMA, r/OpenAI — Aug/Sep 2025

And Hands-On, the Cracks Showed Too

Honest Assessment: Intelligence Gaps

- [Logic Problem Example](#) - Basic reasoning failure
- [Melting App Fail](#) - Physical simulation errors

Takeaway: GPT-5 genuinely moved the frontier on benchmarks — but the router, the abrupt GPT-4o retirement, and real reasoning gaps meant the launch experience didn't match the pitch. Keep both things true at once.

Demo: My Initial Experiments with GPT-5

Interactive Learning Applications

Real GPT-5 Projects (Aug 2025)

Educational Tools Built:

- [Transformers Learning App](#) - Interactive transformer architecture visualization
- [Embeddings Learning App](#) - Hands-on embedding exploration
- [Basketball Coach App](#) - AI-powered sports coaching
- [iPad Drawing App](#) - Touch-optimized creative tool

Advanced Web Applications

Cutting-Edge Implementations

Interactive Experiences:

- [Context Rot Paper → Interactive Experience](#) - Academic paper transformation
- [MediaPipe Music App](#) - Hand tracking music interface

Research Tools:

- [Prompt Experiments](#) - Python scripting and analysis
- [Semantic Reader App](#) - Claude comparison project

Bridge

One Year Later: GPT-5 → GPT-5.6

Version History (Aug 2025 → Jul 2026)

Mostly minor-version iteration — until now?

- **GPT-5** — Aug 2025 (original launch)
- **GPT-5.1** — Nov 2025
- **GPT-5.2** — Dec 2025 (`reasoning_effort` default → none; prompting guide deltas)
- **GPT-5.4** — Mar 2026 (mini + nano variants)
- **GPT-5.5** — Apr 2026 (default `reasoning_effort` reverts to medium)
- **GPT-5.5-instant** — May 2026 (ChatGPT default)
- **GPT-5.6** — Jul 2026 (Sol/Terra/Luna tiers; `reasoning.mode` replaces separate -pro model; Programmatic Tool Calling added to Responses API)

Act II

GPT-5.6: Where We Are Today

July 2026

GPT-5.6 Core Innovations

Same DNA as GPT-5, evolved knobs

- **Frontier Intelligence** — "a new class of intelligence for coding and professional work"
- **Explicit Model Selection** — gpt-5.6-sol, gpt-5.6-terra, gpt-5.6-luna (you choose, instead of a router guessing)
- **Tunable Reasoning** — none → low → medium → high → xhigh → max
- **Enhanced Tool Use** — shell, Skills, Connectors
- **Multimodal** — text, image, audio, video

What Actually Changed Since GPT-5?

Model selection: router guessing main/thinking (and a bad reputation for it, see Act I) → you explicitly pick sol/terra/luna

Reasoning control: coarse tiers → 6-level tunable `reasoning.effort`, incl. pro mode on Sol (no separate -pro model)

Conversation state: manual `previous_response_id` chaining → durable `conversations.create()` object + server-side compaction

New in the API: Programmatic Tool Calling added to the Responses API

This is cumulative refinement, not another leap. The honest question for students: is that a problem?

Model Comparison & Performance

GPT-5.6 Sol — The Current Frontier Model

"A new class of intelligence for coding and professional work" —
OpenAI

- **1,050,000-token** context window; **128,000** max output tokens
- Default `reasoning.effort` is `medium`; supports `none`, `low`, `medium`, `high`, `xhigh`, `max`
- Hardest problems: same model, `reasoning: {"mode": "pro"}` — no separate `-pro` model anymore

GPT-5.6 Model Line (Current Variants)

Per 1M tokens, standard tier

Model	Input	Output	Context
gpt-5.6-sol (alias gpt-5.6)	\$5.00	\$30.00	1,050,000
gpt-5.6-terra	\$2.50	\$15.00	1,050,000
gpt-5.6-luna	\$1.00	\$6.00	1,050,000

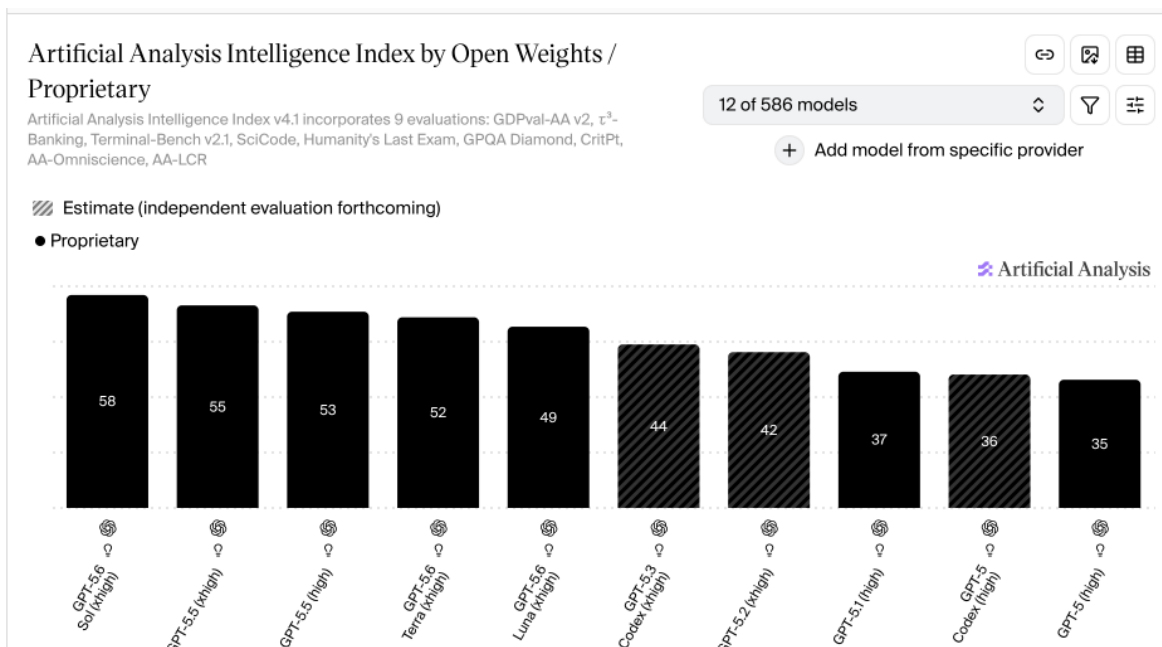
GPT-5 → GPT-5.6: What Changed in the Price Tag

Same context-window ballpark, a very different sticker price

	GPT-5	Sol	Terra	Luna
Input /1M	\$1.25	\$5.00	\$2.50	\$1.00
Output /1M	\$10.00	\$30.00	\$15.00	\$6.00

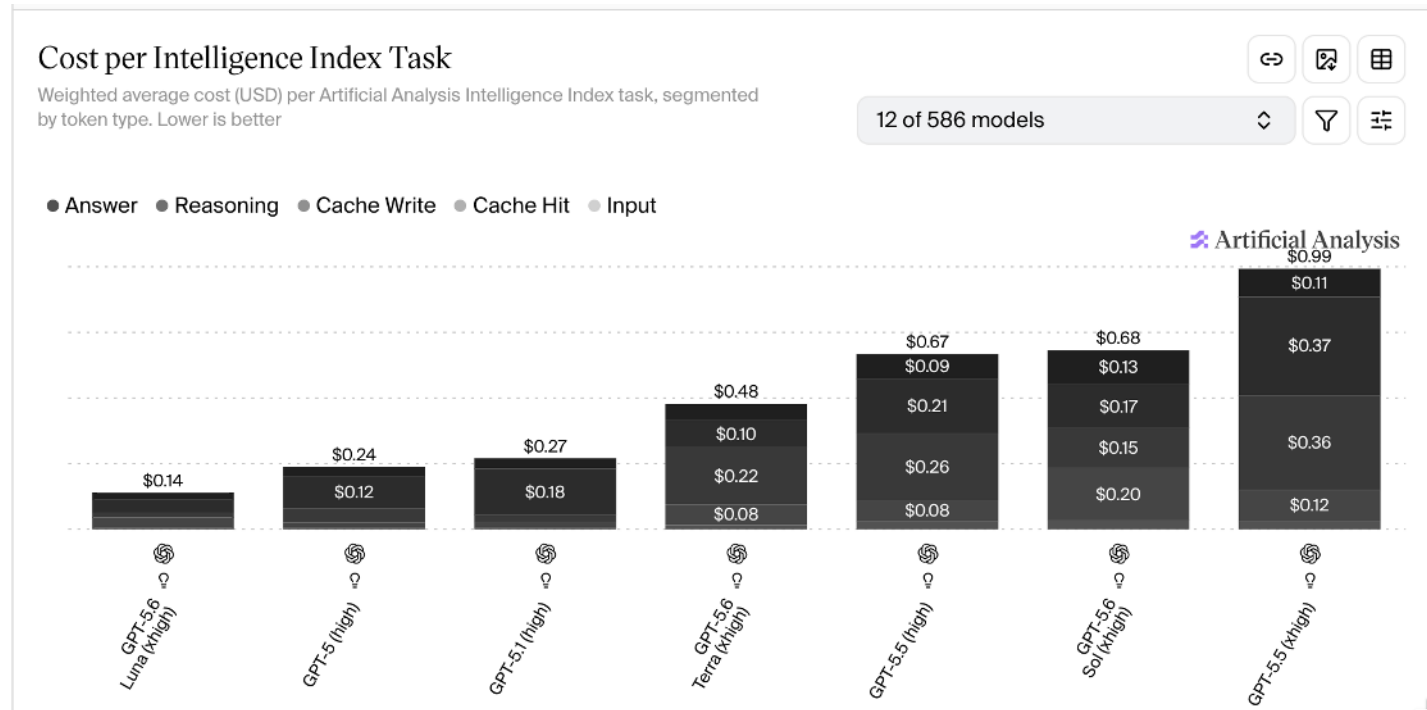
The flagship reasoning tier now costs **roughly 4x more** than GPT-5's single launch price — but a new budget tier (luna) undercuts it. Pricing didn't get cheaper across the board; it split into a spectrum.

GPT-5 → GPT-5.6: The Quality Jump, Measured



Same benchmark suite, same scale, one model generation apart: **GPT-5 (high): 35 → GPT-5.6 Sol (xhigh): 58** on the Artificial Analysis Intelligence Index.

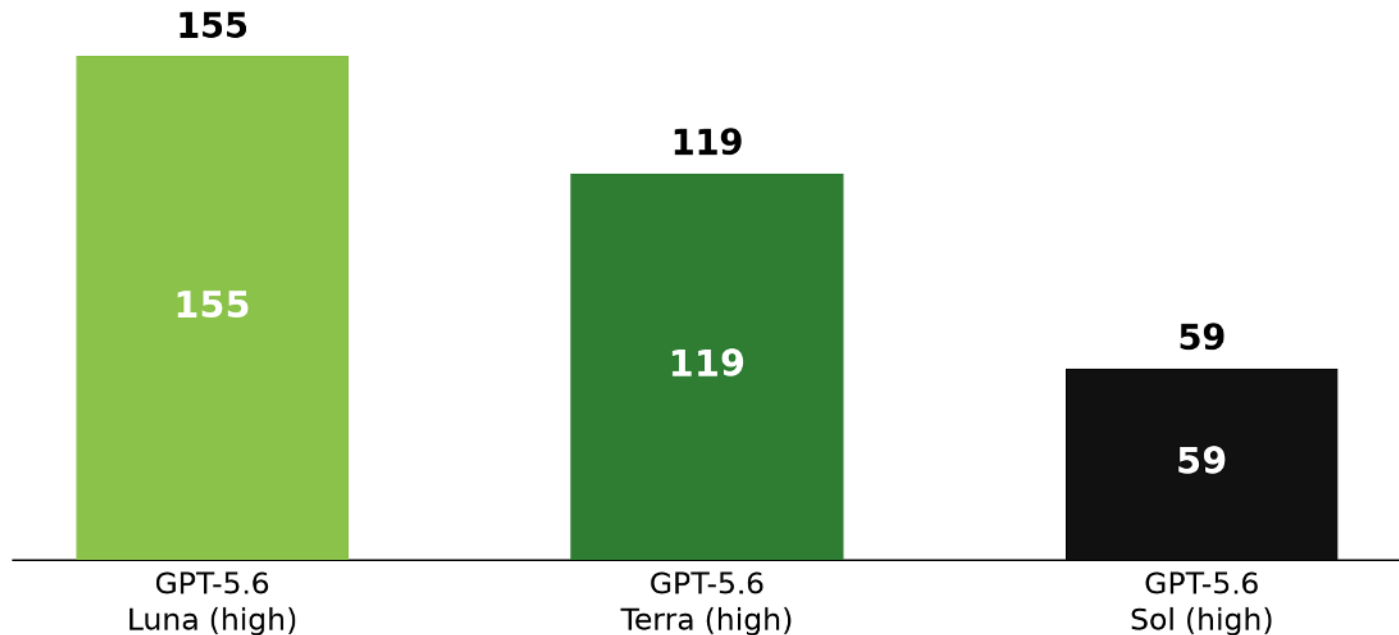
...But at a Real Cost



GPT-5 (high): \$0.24 → GPT-5.6 Sol (xhigh): \$0.68 per Intelligence Index task — roughly 3x. The budget tier tells a different story: **Luna (xhigh) is \$0.14**, cheaper than GPT-5 despite being the newer model.

Output Speed

Output tokens per second · Higher is better



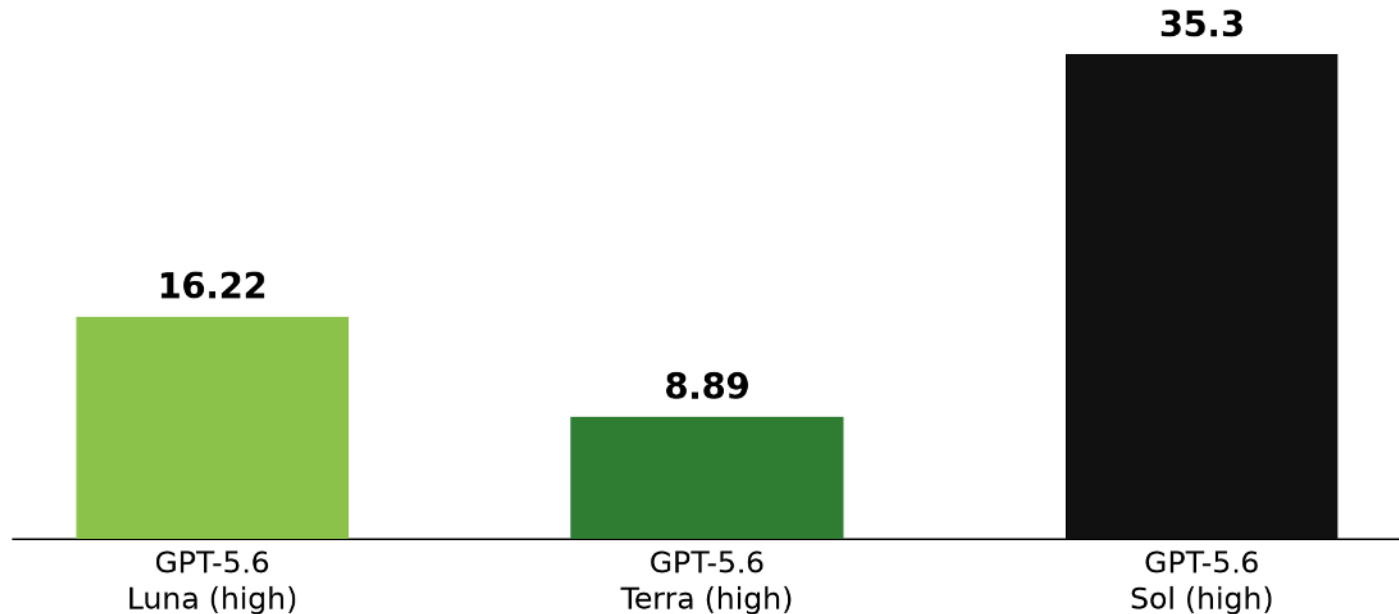
Source: <https://artificialanalysis.ai/> (GPT-5.6 Sol/Terra/Luna, high reasoning effort, fetched 2026-07-25)

GPT-5.6 Luna: 155 tok/s · Terra: 119 tok/s · Sol: 59 tok/s (high reasoning) — the classic speed/quality trade-off, now split across three named tiers instead of one router guess

Source: <https://artificialanalysis.ai/> (GPT-5.6 Sol/Terra/Luna, high reasoning effort), fetched 2026-07-25

End-to-End Response Time

Seconds to output 500 tokens, including reasoning 'thinking' time · Lower is better



Source: <https://artificialanalysis.ai/> (GPT-5.6 Sol/Terra/Luna, high reasoning effort, fetched 2026-07-25)

GPT-5.6 Terra: 8.89s · GPT-5.6 Sol: 35.3s end-to-end (incl. thinking time, 500 tokens)

Source: <https://artificialanalysis.ai/>, fetched 2026-07-25

Honest Assessment: GPT-5.6 Today

Fixed Since GPT-5

- **Router distrust** — gone, because there's no router. You pick the model explicitly.
- **Coarse reasoning control** — 6 tunable levels instead of a hidden switch
- **Abrupt deprecations** — GPT-5.6's own deprecation notices (Agent Builder, Evals, v1/prompts) shipped with a 6-month notice window, unlike the GPT-4o retirement backlash
- **Safety approach** — safe-completions matured, same framework, no regression

Still True (or New) Today

- **Simon Willison**, on the three-tier lineup: *"One of the most confusing aspects of GPT-5.6 is figuring out which model to use at which reasoning effort."*
- **Sol's weakness: fuzzy judgment** — for open-ended architecture or product-tradeoff questions with several defensible answers, still worth running multiple models in parallel
- **Luna's weakness: shallow reasoning** — weaker on long-context retention and nuance; don't reach for it on complex multi-step analysis
- **Terra is the sweet spot for most work** — roughly GPT-5.5-class quality at about half the price

Demo: Getting Started with GPT-5.6

Building with GPT-5.6: API & Integration

Responses API

A new API primitive for agents, combining the simplicity of Chat Completions with the ability to use built-in tools like web search, file search, and connectors.

[Learn more >](#)

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const response = await client.responses.create({
5   model: "gpt-4o",
6   input: "Write a one-sentence bedtime story about a unicorn."
7 });
8 console.log(response.output_text);
```

Conversations API

Before (legacy): manual `previous_response_id` chaining

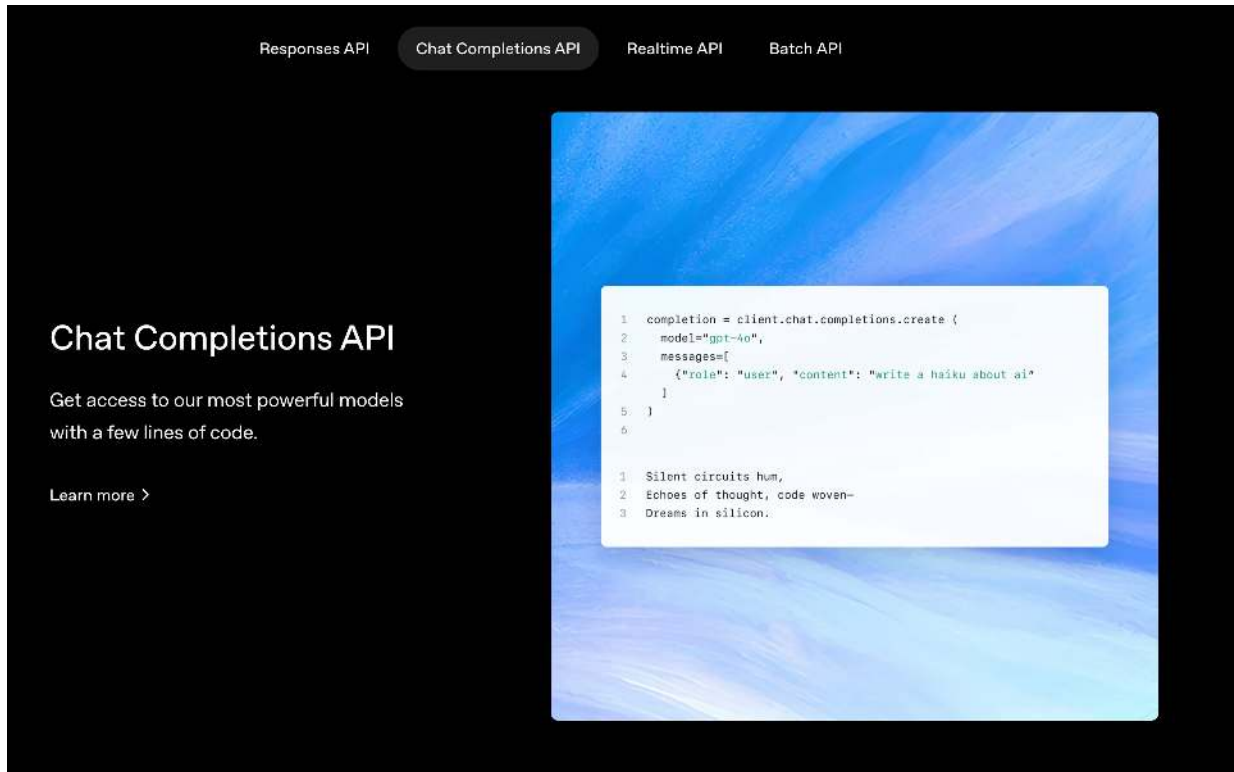
```
r1 = client.responses.create(model="gpt-5.6-sol", input="tell me a joke")
r2 = client.responses.create(
    model="gpt-5.6-sol",
    previous_response_id=r1.id,
    input="explain why this is funny",
)
```

After: a durable conversation object

```
convo = client.conversations.create()
client.responses.create(
    model="gpt-5.6-sol", conversation=convo.id, store=True,
    input="tell me a joke",
)
```

Conversations API — What Else It Gives You

- **Server-side compaction** — `/responses/compact` shrinks context automatically each turn
- **phase parameter** — `commentary` (thinking out loud) vs `final_answer` (the deliverable)



Chat Completions API — legacy, still supported for GPT-5.6
Current recommendation: build new work on the Responses API (previous slide) — keep Chat Completions only for existing integrations

[Responses API](#)[Chat Completions API](#)[Realtime API](#)[Batch API](#)

Realtime API

Build low-latency, multimodal experiences, including speech-to-speech.

[Learn more >](#)

```
1 // Connect to OpenAI's WebSocket server
2 const ws = new WebSocket("wss://api.openai.com/v1/realtime",
  /* options */);
3
4 // Get streaming data from the Realtime model
5 ws.onmessage = (event) => {
6   const realtimeEvent = JSON.parse(event.data);
7   if (realtimeEvent.type === "response.audio.delta") {
8     // handle audio stream
9   }
10 };
11
12 // Send audio prompts to the model
13 ws.send(JSON.stringify({
14   type: "input_audio_buffer.append",
15   audio: "<base64 encoded audio>"
16 }));
17
18 // Handle WebSocket events
19 ws.onopen = () => { /* */ };
20 ws.onerror = () => { /* */ };
21 ws.onclose = () => { /* */ };
```

[Realtime API](#)

[Responses API](#)[Chat Completions API](#)[Realtime API](#)[Batch API](#)

Batch API

Run asynchronous workloads for 50% of the cost over 24 hours.

[Learn more >](#)

```
curl https://api.openai.com/v1/batches \
-H "Authorization: Bearer $OPENAI_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "input_file_id": "file-abc123",
  "endpoint": "/v1/chat/completions",
  "completion_window": "24h"
}'
```

[Batch API](#)

Agents SDK

The 7 Agents SDK Primitives

- **Agent** — the LLM config: instructions + tools; plans, calls tools, keeps state
- **Runner** — executes the agent loop; manages streaming, state, result handling
- **Tools** — function tools (`@function_tool`), hosted tools, and MCP servers
- **Handoffs** — delegate to specialist agents; you control which agent replies
- **Guardrails** — input/output validation; pause/block risky ops, add human review
- **Sessions** — automatic conversation-history management, resumable across turns
- **Tracing** — built-in run tracking to view, debug, and optimize workflows

Agents SDK — Minimal Example

Define an Agent, run it with streaming

```
from agents import Agent, Runner

agent = Agent(
    name="Assistant",
    model="gpt-5.6-sol",
    instructions="Help users with their tasks",
)

result = Runner.run_streamed(agent, "Summarize today's standup notes")
async for event in result.stream_events():
    print(event)
```

Agents SDK — Handoffs

Triage agent delegates to a specialist

```
from agents import Agent, Runner

refunds = Agent(
    name="Refunds Specialist",
    model="gpt-5.6-sol",
    instructions="Handle refund requests end to end.",
)

triage = Agent(
    name="Triage",
    model="gpt-5.6-terra",
    instructions="Route the user. Hand off refund issues to the specialist.",
    handoffs=[refunds],          # explicit delegation
)

result = Runner.run_sync(triage, "I want a refund for order #4821")
print(result.final_output)
```

Q&A & Break

GPT-5.6 Prompting Guide Complete Breakdown

Agentic Workflow Predictability Spectrum

GPT-5 operates within a spectrum of:

Autonomous
Decision Making



Strict
Instruction
Following



Handles ambiguous tasks
with intelligent exploration

Executes exactly as specified
when given clear constraints

Prompting guidance changes almost every model generation. The next few slides lead with **current GPT-5.6 practice** — older patterns are noted only as a small "previously" aside, so you can spot and unlearn outdated mental models instead of carrying them forward.

Verbosity Is a Global Parameter

Now: control output length with the API-level `text.verbosity` parameter (low/medium/high) — a single global switch, not something you write into the prompt.

```
text={"verbosity": "low"}
```

Why: set once, applies consistently — it doesn't compete with your actual task instructions for the model's attention inside the prompt.

Previously: this needed an in-prompt `<output_verbosity_spec>` block spelling out sentence/bullet counts. If that's the mental model you learned, drop it — the guide moved this back to the API layer.

State the Constraint, Skip the Ceremony

Now: one plain sentence carries the same weight a whole fenced block used to need — the model follows prompt contracts closely, so ceremony doesn't add safety margin anymore.

Implement only what's requested – no extra features, components, or UX embellishments.

Why it's enough now: GPT-5.6 treats this as a real constraint to satisfy, not a suggestion to weigh against other instructions.

Previously: the same guidance needed a dedicated `<scope_discipline>` XML block to fight over-coding. The constraint was always correct — only the packaging is obsolete.

Trust the Default Grounding

Now: re-grounding on long inputs and flagging unstated assumptions or ungrounded claims are things the model now does reliably on its own — you generally don't need to instruct it to outline, re-state constraints, or self-check before answering.

Reserve explicit instructions for genuinely unusual cases: adversarial or contradictory source material, or domains where a wrong answer is costly enough that you want the check made visible in the output.

Previously: this needed explicit step-by-step re-grounding and uncertainty-self-check instructions written into every long-context or high-risk prompt. GPT-5.6 does this by default — verify occasionally, don't over-prompt for it.

Agentic Eagerness — Turning It Down

Now: `reasoning.effort` is still the real lever — set it low for fast, narrow, low-latency tasks.

```
reasoning={"effort": "none"}
```

Stopping conditions still matter — say them once, plainly:

Search only in `src/`. Stop once you find the function or after 2 searches.

Benefits: faster responses, more predictable behavior, lower token usage.

Previously: this same goal needed a full `<context_gathering>` XML template. GPT-5.6's guide names templates like that as trim-worthy — the plain sentence above does the same job.

Agentic Eagerness — Turning It Up

Now: raise `reasoning.effort` for tasks that need real persistence through to completion.

```
reasoning={"effort": "high"} # default is "medium"
```

The model persists on its own now — it keeps working through a task and verifies edge cases without being told to, so a reminder block isn't doing any work anymore.

Tool-calling updates: a brief one-liner when starting a new phase or changing plan is enough — not a running narration of every step.

Previously: this needed an explicit `<persistence>` block plus instructions for thorough step-by-step progress narration. GPT-5.6's guide flags both as things to trim; keep `reasoning.effort` as the real lever.

Performance Optimization Techniques

Context Reuse & Efficiency

Leverage Previous Reasoning Traces

- Reuse context across turns instead of rebuilding history by hand
- **Current best practice:** durable conversation objects
(`conversations.create() + store=True`) over manual `previous_response_id` chaining

Demo: Context Reuse & Efficiency

Run the Responses API context-reuse and conversation-state examples hands-on.

`notebooks/0.0-introduction-openai-responses-api.ipynb`

The Hidden Cost of Bad Prompts

- GPT-5's stricter instruction following amplifies poor prompts
- Contradictory instructions lead to inefficient Reasoning
- Always resolve instruction hierarchy conflicts

Example: Resolving Conflicts

✗ "Be concise but provide detailed explanations"

✓ "Provide a 2-sentence summary, then detailed bullet points"

GPT-5.6 Coding Excellence

Optimal Framework Stack

Frontend Frameworks:

- Next.js (TypeScript)
- React
- HTML

Styling/UI:

- Tailwind CSS
- shadcn/ui
- Radix Themes

Icons & Animation:

- Material Symbols
- Heroicons, Lucide
- Motion library

Typography:

- Sans Serif, Inter
- Geist, Mona Sans
- IBM Plex Sans, Manrope

Ask for the Outcome, Not the Self-Review

Now: state the quality bar once, as an outcome — the model already applies this kind of review internally without being asked to narrate a score.

Make sure components are reusable, errors are handled, and the implementation is reasonably performant before finishing.

Previously: this needed a <self-reflection> block asking the model to score itself 1–10 on each dimension and explain improvements. That scored checklist is exactly the kind of process narration GPT-5.6's guide says to trim.

Concrete Constraints, Not Absolute Words

Now: keep the specific, measurable rules — drop the "always"/"never" framing around them.

Typography: 4-5 sizes max. Spacing: multiples of 4.

Color: 1 neutral base + up to 2 accents.

Loading state: skeleton placeholders. Prefer Radix/shadcn for built-in accessibility.

Why: GPT-5.6 follows prompt contracts closely enough that hard numbers and named defaults do real work on their own — wrapping them in "Always..."/"Never..." adds friction without adding compliance.

Previously: the same rules were framed inside a `<ui_ux_best_practices>` block full of "Always" directives. The constraints were correct then and still are — only the absolute-word packaging is what GPT-5.6's guide flags as historical.

Ecosystem Support

- **GitHub Copilot:**
 - Available across all paid plans
 - VS Code, GitHub.com, GitHub Mobile
 - Enhanced multi-file changes/refactors
- **Cursor Feedback:**
 - "Smartest coding model we've used"
 - Optimized for interactive development
- **Third-Party Tools:**
 - Windsurf integration Codex CLI optimization
 - Fine-tuned for agentic coding
- **gpt-5.6-sol:**
 - Absorbed the dedicated-coding-model role gpt-5.3-codex used to hold
 - Real-world task training
 - Complex independent task handling

Recommended Prompting Patterns

1. Brief Thought Process Summary

"Start your response with 2-3 bullet points summarizing your reasoning approach before the full answer"

2. Tool-Calling Preambles *(historical — GPT-5.6's guide says trim this to a brief one-liner, not a running narration)*

"Before each tool use, explain: what you're doing, why, and what you expect to find"

3. Disambiguated Tool Instructions

"Search for 'class UserAuth' in authentication files only, stop after finding the implementation, not just imports"

4. Planned Execution at Minimal Reasoning

"First, outline your approach in 3 steps. Then execute each step completely before moving on."

Markdown Formatting & Output Control

Professional Output Formatting

Semantic Markdown Usage:

Use Markdown ****only where semantically correct****:

- ``inline code`` for functions, variables
- ````code fences```` for code blocks
- Lists and tables for structured data
- $\backslash($ and $\backslash)$ for inline math
- $\backslash[$ and $\backslash]$ for block math

GPT-5.6 Prompting Guide: The Philosophy Flips

Current Best Practice: Trim, Don't Scaffold

GPT-5.6's prompting guide reverses earlier advice: outcome-first, not scaffolding-first.

Trim:

- XML persistence blocks, repeated style rules that don't change behavior
- Context-gathering templates, self-graded checklists, process narration
- Absolute directives ("always" / "never")

Keep:

- Outcome statements, success criteria, stopping conditions
- Hard, concrete constraints (specific numbers, specific formats)

Why the Shift: Measured Impact

Measured impact of trimming: +10–15% eval scores, –41–66% tokens, –33–67% cost

New tools instead of prompt scaffolding: a global `text.verbosity` parameter, and Programmatic Tool Calling (offload filtering/batching to code, not model judgment)

Why the reversal happened: GPT-5.6 "follows prompt contracts closely" — conflicting or redundant rules now cause instability (the model burns reasoning tokens reconciling contradictions) rather than adding safety margin the way they did on less-steerable 2025-era models.

Demo: Prompting Guide for GPT-5.6 - Hands-on

Demo: Prompt Optimizer Tool

Q&A & Break

Demo: Building GPT-5.6 Powered Apps

Q&A & Break

Key Takeaways

Closing the Two-Act Arc

- **GPT-5 proved the leap:** a unified model, frontier benchmarks across math/coding/multimodal, and safe-completions as the new safety standard — with real rollout scars (router backlash, abrupt GPT-4o retirement)
- **GPT-5.6 refined it:** the router became explicit model selection (sol/terra/luna), reasoning went from a hidden switch to 6 tunable levels, and durable Conversations replaced manual chaining — cumulative, not revolutionary
- **What's still hard:** picking the right tier/effort combo, Sol's fuzzy judgment on open-ended tradeoffs, Luna's shallow reasoning on complex tasks
- **Comprehensive API ecosystem** with built-in tools, Agents SDK, and MCP integration ties both eras together

What NOT to Use (Deprecated as of June 2026)

- **Agent Builder** (visual workflow tool) — sunset **Nov 30, 2026**
 - Migrate to the **Agents SDK** or ChatGPT Workspace Agents
- **Evals platform** — **read-only Oct 31, 2026**, full shutdown **Nov 30, 2026**
 - Move to the SDK's tracing → eval loop, or code-based evals
- **Reusable prompt objects** (v1/prompts) — shutdown **Nov 30, 2026**
 - Move prompt content into application code

These were announced deprecated **Jun 3, 2026** — don't build new workflows on them.

Next Steps & Resources

Continue Your GPT-5.6 Journey

Official Documentation

- [GPT-5.6 System Card](#) (Jul 9, 2026)
- [GPT-5.6 Model Guidance & Prompting Guide](#)
- [OpenAI API Documentation](#)

Hands-on Practice

- [Prompt Optimizer \(Playground\)](#)
- [Programmatic Tool Calling Guide](#)
- [GPT-5.6 Sol Model Reference](#)

Connect With Me



[Course materials](#)



[LinkedIn](#)



[Twitter/X - @LucasEnkrateia](#)



[YouTube - @automatalearninglab](#)



Email: lucasenkrateia@gmail.com